Index

SR No	Date	Title	Sign
1.		Implementing Substitution and Transposition Ciphers	
2.		RSA Encryption and Decryption	
3.		Message Authentication Codes	
4.		Digital Signatures	
5.		Key Exchange using Diffie-Hellman	
6.		Malware Analysis and Detection	
7.		Firewall Configuration and Rule-based Filtering	

Practical 1:

Implementing Substitution and Transposition Ciphers

Aim: To study and implement the Substitution and Transposition Ciphers

1. **Caesar Cipher Code (*python*):**

```
def encrypt(text,s):
    result = ""

    for i in range(len(text)): char = text[i]

      result += chr((ord(char) + s-65) % 26 + 65)
      result += chr((ord(char) + s - 97) % 26 + 97)

    return result

text=input(" Enter the text to encrypt: ")
s = 3; s = str(s)

print("Text : " + text)
print( "Cipher: " + encrypt(text,s))
```

```
$python ts.py
Enter the text to encrypt: Hello World
Text : Hello World
Cipher: KeNhUoUoXrWqZtXrAuUoMg
```

2. **Railfence Cipher Code (*python*):**

```
def RailFence(txt):
    result=""
    for i in range(len(string)):
        if (i%2==0): result+=string[i]
    for i in range(len(string)):
        if (i%2!=0): result += string[i]
    return result

string=input("enter a string")
print(RailFence(string))
```

```
$python ts.py
Enter a string: Hello World
HloWrdel ol
```

Practical 2:

RSA - Encryption and Decryption

Aim: To study and implement the RSA Encryption and Decryption

Code:

```
from Crypto.PublicKey import RSA
from Crypto.Cipher import PKCS1_OAEP
import binascii

keyPair = RSA.generate(1024)
pubKey = keyPair.publickey()

print(f"Public key: (n={hex(pubKey.n)}, e={hex(pubKey.e)})")
pubKeyPEM = pubKey.exportKey()
print(pubKeyPEM.decode('ascii'))

print(f"Private key: (n={hex(pubKey.n)}, d={hex(keyPair.d)})")
privKeyPEM = keyPair.exportKey()
print(privKeyPEM.decode('ascii'))

msg = 'Hello World'.encode()
encryptor = PKCS1_OAEP.new(pubKey)
encrypted = encryptor.encrypt(msg)
print("Encrypted:", binascii.hexlify(encrypted))

decryptor = PKCS1_OAEP.new(keyPair)
decrypted = decryptor.decrypt(encrypted)
print("Decrypted:", decrypted.decode())
```

Output:

```
(insPractical) $python ts.py
Public key: (n=0x938c145c9d5c58c135ed47f66e99adfa4199d1c2d5c88e41ef25de6d8d11469dcc4fb0cca5a64ff8a8b78f4d3b
da302ce561fa63b99d52b63c386370eb550dfe04e27c791d16806c342f164370f33c3f0665dd17c78f43dfe40a9d0c46219703281e3
7e951e4fbc351a11bcfcf9c10b8450e06d7b90f018a2c1dc28c9b301e51, e=0x10001)
-----BEGIN PUBLIC KEY-----
MIGfMA0GCSqGSIb3DQEBAQUAA4GNADCBiQKBgQCTjBRcnVxYwTXtR/Zuma36QZnR
wtXIjkHvJd5tjRFGncxPsMylpk/4qLePTTvaMCzlYfpjuZ1Stjw4Y3DrVQ3+BOJ8
eR0WgGw0LxZDcPM8PwZl3RfHj0Pf5AqdDEYhlwMoHjfpUeT7w1GhG8/PnBC4RQ4G
17kPAYosHcKMmzAeUQIDAQAB
-----END PUBLIC KEY-----
Private key: (n=0x938c145c9d5c58c135ed47f66e99adfa4199d1c2d5c88e41ef25de6d8d11469dcc4fb0cca5a64ff8a8b78f4d3
bda302ce561fa63b99d52b63c386370eb550dfe04e27c791d16806c342f164370f33c3f0665dd17c78f43dfe40a9d0c46219703281e
37e951e4fbc351a11bcfcf9c10b8450e06d7b90f018a2c1dc28c9b301e51, d=0x76f7bbc89c19447730198d5766565906dab396d94
25c8902763c52872d4cd60e3b6f0609fc86dd1f379574e11176c6e16c6853e6ea585a6e15f5b6e34032947f1930c973e7dc53953b80
cac4bab4583f962eb5e8b6b8baf2b4c18c53da7a4d87cc6e1dfe902dbe9ce5f5d3d20dac2586209671eaa7267ef631bb63b237e0b71
)
-----BEGIN RSA PRIVATE KEY-----
MIICXAIBAAKBgQCTjBRcnVxYwTXtR/Zuma36QZnRwtXIjkHvJd5tjRFGncxPsMyl
pk/4qLePTTvaMCzlYfpjuZ1Stjw4Y3DrVQ3+BOJ8eR0WgGw0LxZDcPM8PwZl3RfH
j0Pf5AqdDEYhlwMoHjfpUeT7w1GhG8/PnBC4RQ4G17kPAYosHcKMmzAeUQIDAQAB
AoGAB297vInBlEdzAZjVdmVlkG2rOW2UJciQJ2PFKHLUzWDjtvBgn8ht0fN5V04R
F2xuFsaFPm6lhabhX1tuNAMpR/GTDJc+fcU5U7gMrEurRYP5Yutei2uLrytMGMU9
p6TYfMbh3+kC2+nOX109INrCWGIJZx6qcmfvYxu2OyN+C3ECQQDBhCrDzhEGtPGE
iMzKq2BXmWnOMCO0u8WyQQuhlwo2mrMaSmLhIB6bG+dapKVaZOrG253UgW+1F3xQ
gmTGqoVnAkEAwzApBFcBDa5MlaU1GT2JEzptr4B8hYsYhDcC5UbYWHCZcZWQWF7I
87y25BAhr+e1ltzZc/R26Rea/qbfP2XzhwJBAIcEDMZSluSECGI9BA1/ZpeXzQ9z
3OWzgOquoUvb0t9oLf/w2G5T96am4FMIvumDsZ6PCd6DdIn32d9eKqvFu48CQDyO
u5sSF0yUUER5bkD3e4jQruktdvTD1AZpPue9lC/UN3crDQ6t0bD1UtTtQFZgEFHP
EVRNe3sUDM2HHmqACg8CQFHuhzxkzLIZMaVHm38BET1rGbJVDYvIbBUek/CeP5/I
CI7CHHj49+JnjvEOmPDzZ0zVmG9bv3dSEdC3Iaj65r4=
-----END RSA PRIVATE KEY-----
Encrypted: b'0cf97cc3437ccaef1a44b68c256c197a24b50f940c5d8ca96e963c2943ce44782866d8a03c742d477af89e01fe7c60
0b538705991720e07f0c58998d40bed1e220874c5c8b8759b2ab2df9863b92ffc3a65377c6f77c01b4aba3f6c1c53b00488a65ed01c
4506f8dadee07ff846bcd7c6950b386418e97a51b8cb001dbc636e9'
Decrypted: Hello World
```

Practical 3:

Message Authentication Codes (MAC)

Aim: To study and implement the Message Authentication Code for ensuring the message integrity and authenticity

Code: (*SHA1 Algorithm***)**

```
import hashlib
str = input("Enter the value to encode: ")
result = hashlib.sha1(str.encode())
print("The hexadecimal equivalent of SHA1 is: ", result.hexdigest())
```

Output:

```
(insPractical) $python ts.py
Enter the value to encode: Hello World
The hexadecimal equivalent of SHA1 is:  0a4d55a8d778e5022fab701977c5d840bbc486d0
```

Practical 4:

Digital Signatures

Aim: To study and implement the Digital Signature algorithm.

Code: (*RSA Algorithm*)

```
from Crypto.Signature import PKCS1_v1_5
from Crypto.Hash import SHA256
from Crypto.PublicKey import RSA
from Crypto import Random

def generate_signature(private_key, message):
    key = RSA.importKey(private_key)
    hashed_message = SHA256.new(message.encode('utf-8'))
    signer = PKCS1_v1_5.new(key)
    signature = signer.sign(hashed_message)
    return signature
def verify_signature(public_key, message, signature):
    key = RSA.importKey(public_key)
    hashed_message = SHA256.new(message.encode('utf-8'))
    verifier = PKCS1_v1_5.new(key)
    return verifier.verify(hashed_message, signature)

random_generator = Random.new().read
key_pair = RSA.generate(2048, random_generator)
public_key = key_pair.publickey().export_key()
private_key = key_pair.export_key()
message = "Hello, World!"
signature = generate_signature(private_key, message)
print("Generated Signature:", signature)
is_valid = verify_signature(public_key, message, signature)
print("Signature Verification Result:", is_valid)
```

Output:

```
$ python ts.py
Generated Signature: b'\x8c\'\x94\xf3\xa3G\xe6\xdeu\xc2\x88\xab\xad:PB\x83:\x82\xbe\xa7.\xac\xd2A!\xd6\x98\
xc5\xce!\xf0\x0c\xe6\xc2\xbe\xe2\x17\xd8\xb3h\x99\xbd+\x1dK\x8c\x03l\xbd\xf4\xe5\xe2\xde\xac\x86\xc70\xbdg\
xbb\xe9\tU#_\xa6\x1a\x07!\xc5\x98s]\xc4\x85q\xf8w;\x9c\x14\\\x18\xd6[\xd7\xf68\x0c7\xa0>\xfb4"\xc5\x8d\x96\
x19\x83\xa80\\S#J\xb2\xf4\xdbe\xfe\x89\xdc\x00\x8e+\xcdV\xe2\xcf\xb4\xeb\xd5x\x16\xa5\xc6\x88\x88\xf4\xd6\x
02BAo\x19\x03\xbd\x7f%\xf6\x0b\xbd\xc7]\x8f\xeb\\yN\xbf*\x93\xf3\x81[\x10d\xb5\xf5\xeaC\xd3\xa6\xa2\xa9*{j\
xf6\x00\xf4\x92\xe7T\n\xcf\xd9\xf6[\x92\x9f\xd4mq\t\xc1\x85\xf7TVZ-\x18\x88\xbf\x81e\x9e\xd0\x10\xd56&\x81\
xe2\x07n\x0f\x89\xf4c\xc7\x11f#NE\xb4\x856\xbc%s}\x98\x1e}\x1cC\x80\xdc\xdc\xf1\xa2\xfd\x92\x0e\x92k\xd9\xb
c\x84\x1a\xc1\xdf\xbf\xa8P\xa4\xb9W\x86\xd6\xe9'
Signature Verification Result: True
```

Practical 5:

Key Exchange using Diffe-Hellman

Aim: To study and implement the Diffe-Hellman key exchange algorithm for secure exchange of keys between two entities.

Code:

```
from random import randint

P = 23
G = 9
print('The Value of P is :%d'%(P))
print('The Value of G is :%d'%(G))

a = 4
print('Secret Number for Alice is :%d'%(a))
x = int(pow(G,a,P))

b = 6
print('Secret Number for Bob is :%d'%(b))
y = int(pow(G,b,P))

ka = int(pow(y,a,P))
kb = int(pow(x,b,P))

print('Secret key for the Alice is : %d'%(ka))
print('Secret Key for the Bob is : %d'%(kb))
```

Output:

```
$ python ts.py
The Value of P is :23
The Value of G is :9
Secret Number for Alice is :4
Secret Number for Bob is :6
Secret key for the Alice is : 12
Secret Key for the Bob is : 12
```

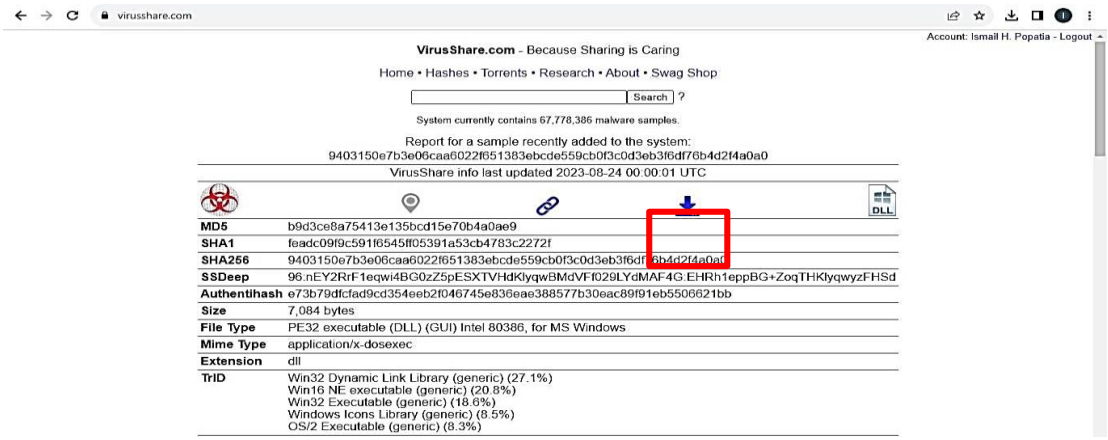
Practical 6:

Malware Analysis and Detection

Aim: To Detect and Analyze Malware (Clean Samples)

Steps:

1. Select the website https://www.virusshare.com for downloading the clean sample of Malware (an account needs to be created for the same). Any other source can be selected to download the Malware (clean sample and authorised site)

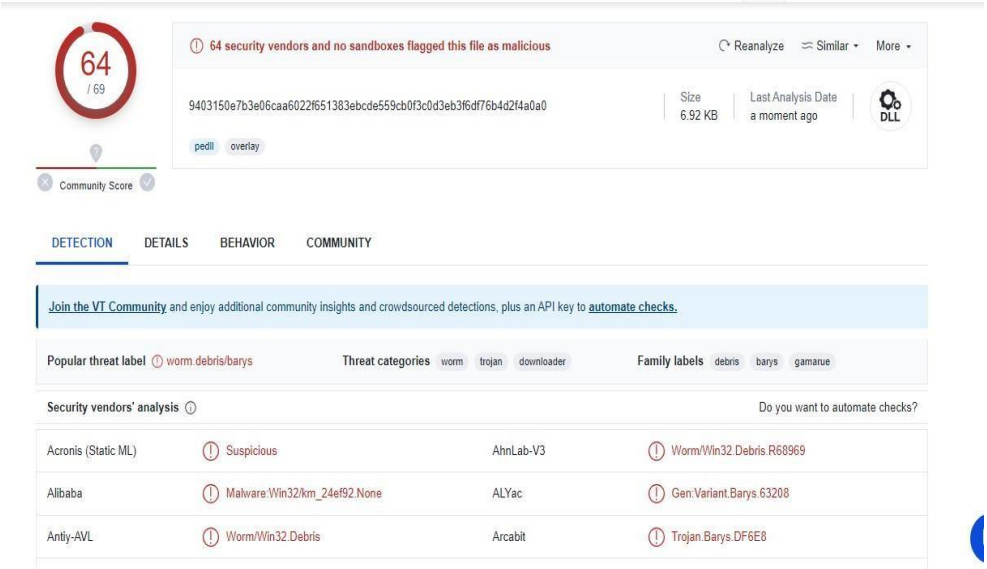


2. By clicking the above download icon the Malware gets downloaded in ZIP format.
3. For unzip the password is “infected”, there is no need to unzip the file, we create a folder “Malware” on desktop and save the file in the folder
4. In order to analyze the Malware, we select the website www.virustotal.com



5. Click on “Choose File” and select the file from the location (ZIP file will do, if asks for password enter infected)

6. We get the following after the upload is complete



7. We interpret the following findings

a) 64 security vendors out of 69 flagged this file as malicious



b) The detection tab shows the threats-type which were flagged by the vendors for e.g

c) The details tab gives the following information

i. Basic properties
ii. History
iii. Compiler products
iv. Header
v. Sections
vi. Imports
vii. Exports
viii. Overlays

d) The Behavior tab gives the following information

i. Activity summary
ii. MITRE ATT&CK Tactics and Techniques
iii. Behavior Similarity Hashes
iv. Process and service actions

Practical 7:

Firewall Configuration and Rule- based Filtering

Aim: To configure and test firewall rules to control network traffic, filter packets based on specified criteria, and protect network resources from unauthorized access.

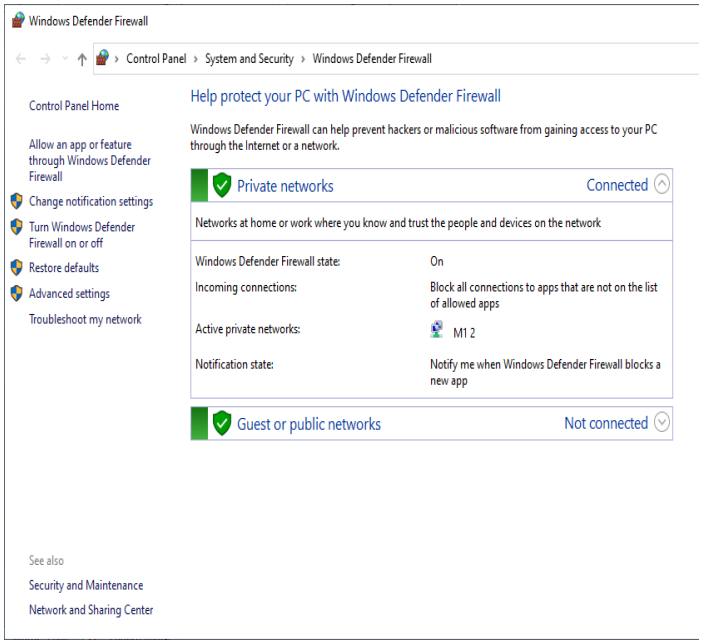
Note the following well-known port (Can be changed as per need)

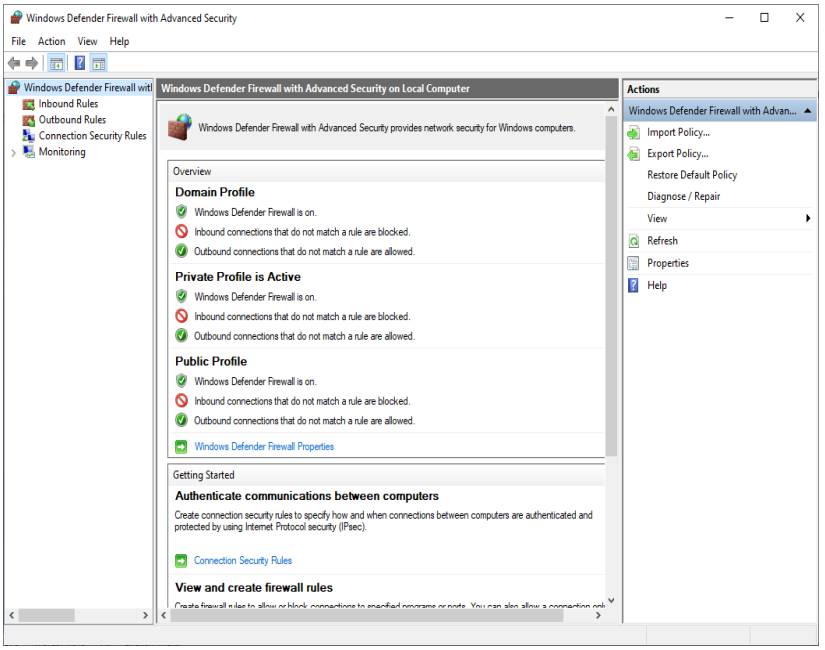
Port Number	Protocol	Application
20	TCP	FTP data
21	TCP	FTP control
22	TCP	SSH
25	TCP	SMTP
53	UDP, TCP	DNS
80	TCP	HTTP (WWW)
110	TCP	POP3
443	TCP	SSL

Part 1: Blocking the HTTP and HTTPS (Port 80 and Port 443) using the Firewall

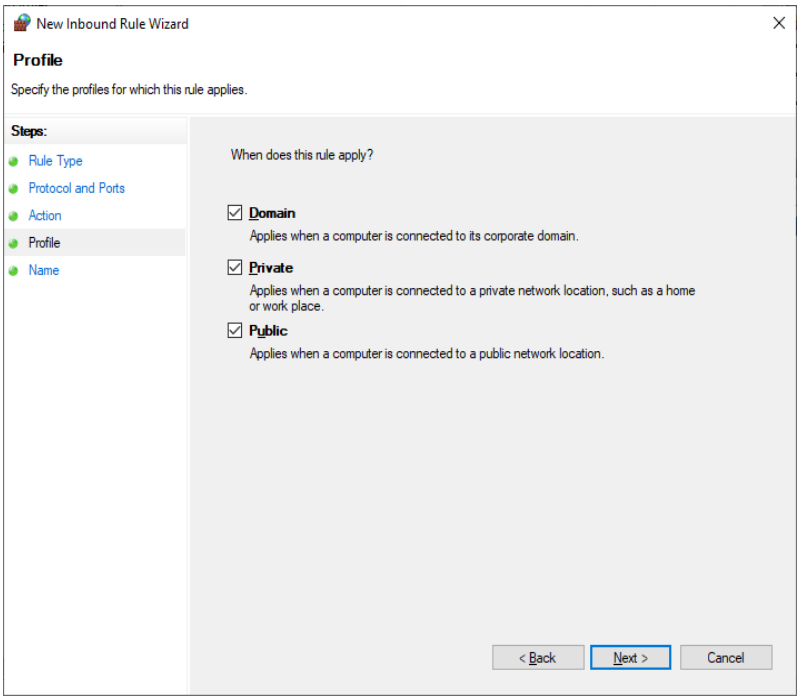
Steps:

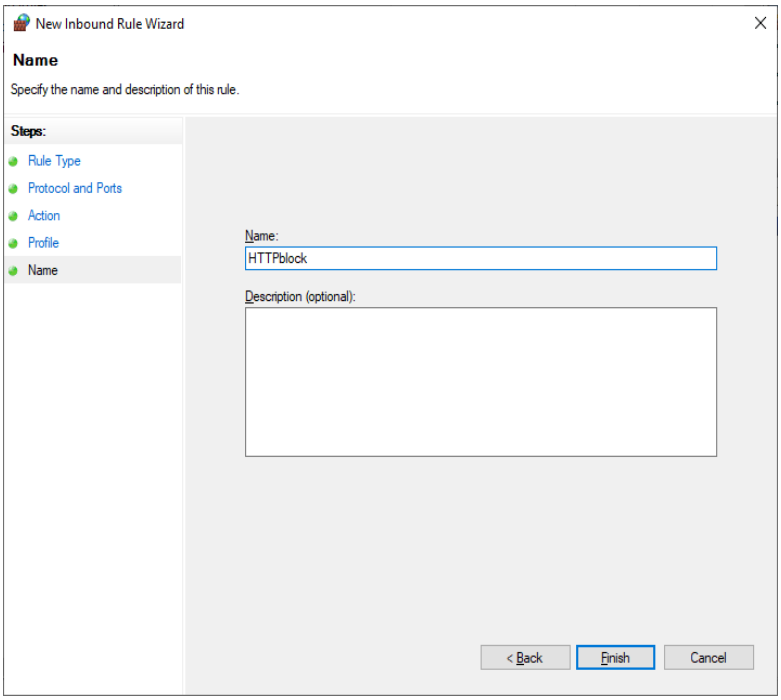
1. We access any website through the browser and confirm that the HTTP/HTTPS protocols are working.
2. We open ‘Windows Defender Firewall’ then click on ‘Advanced settings’



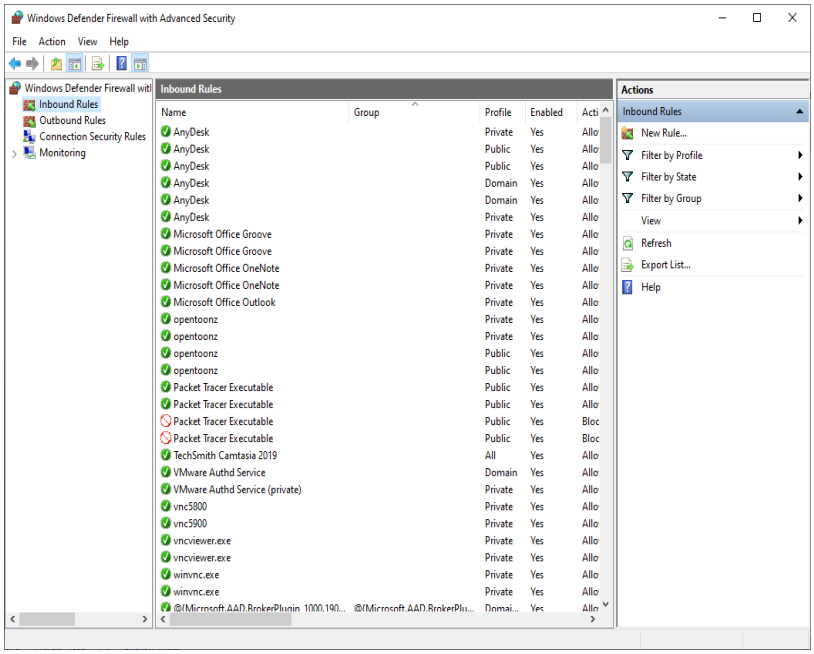


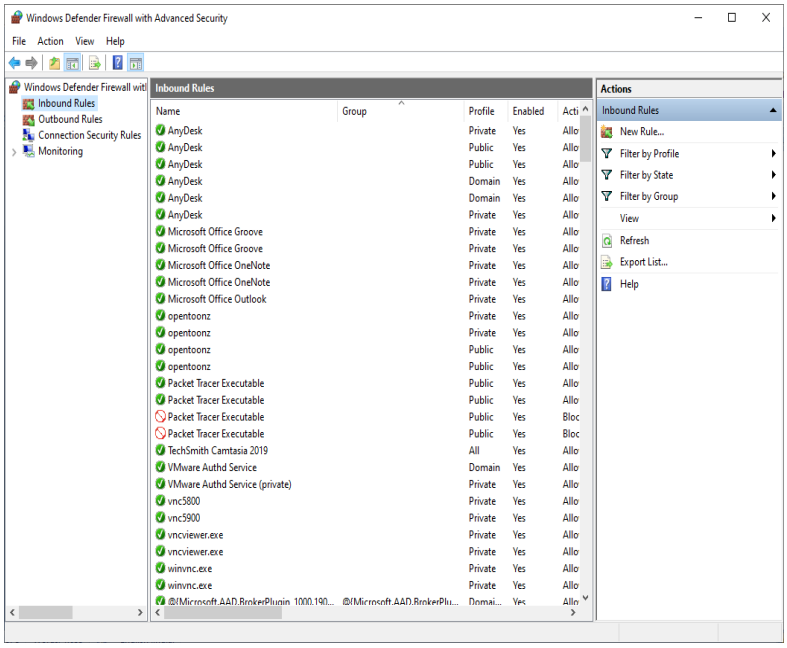
3. Next we click on ‘Inbound Rules’ and then on ‘New Rule’





4. Select the radio button ‘Port’ and click ‘Next’ and enter the following in first figure. After clicking the ‘Next’ button we need to name the rule and click finish.

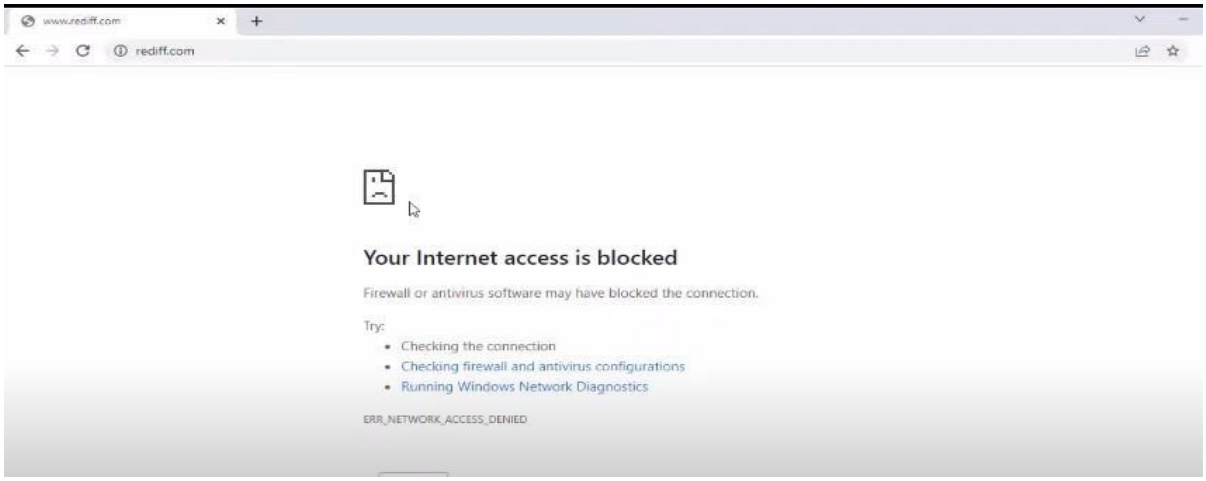




5. The Inbound rule is added



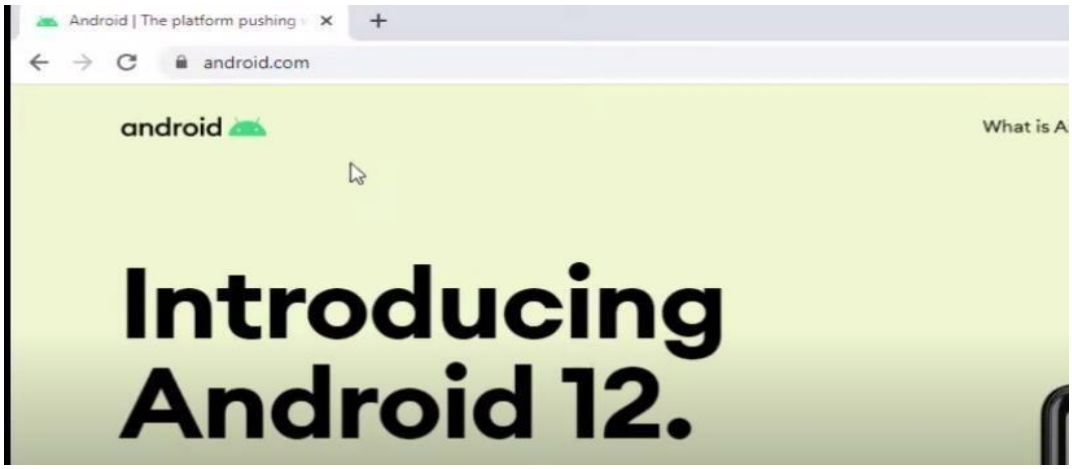
6. We repeat all the above steps for creating ‘Outbound Rules’, and then try to access the internet. We see that the accessed is blocked



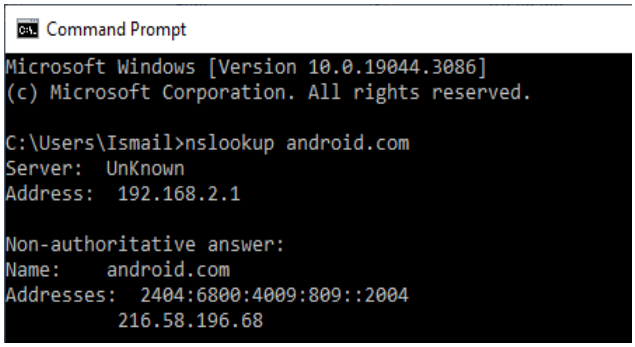
Part 2: Blocking the website www.android.com

Steps:

1. **We open the browser and access the website, which is now accessible**

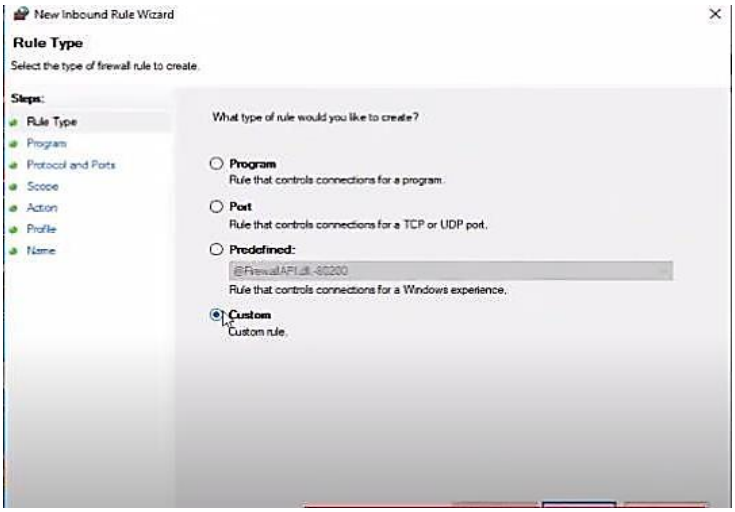


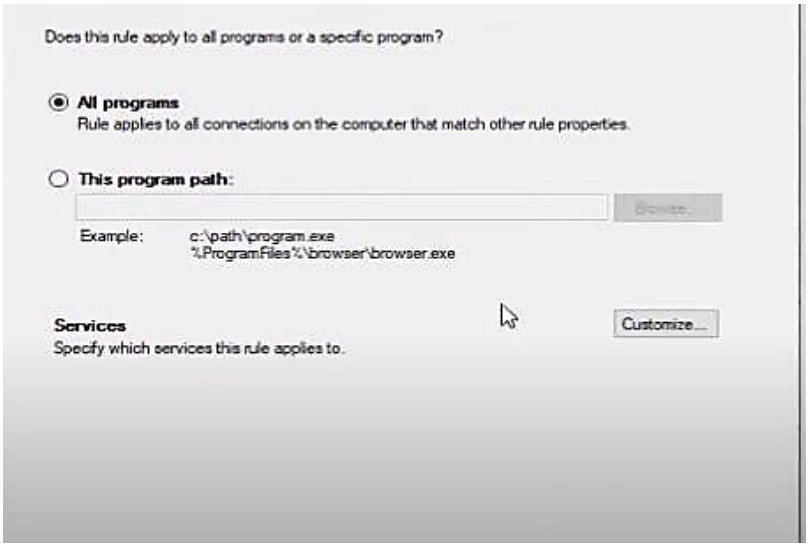
2. **We find the IP addresses of the website using the following command and save the IP addresses**



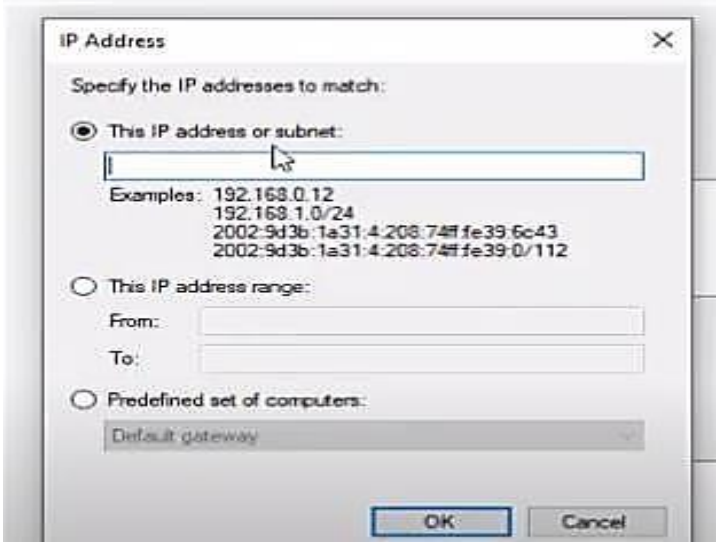
IPv4	216.58.196.68
IPv6	2404:6800:4009:809::2004

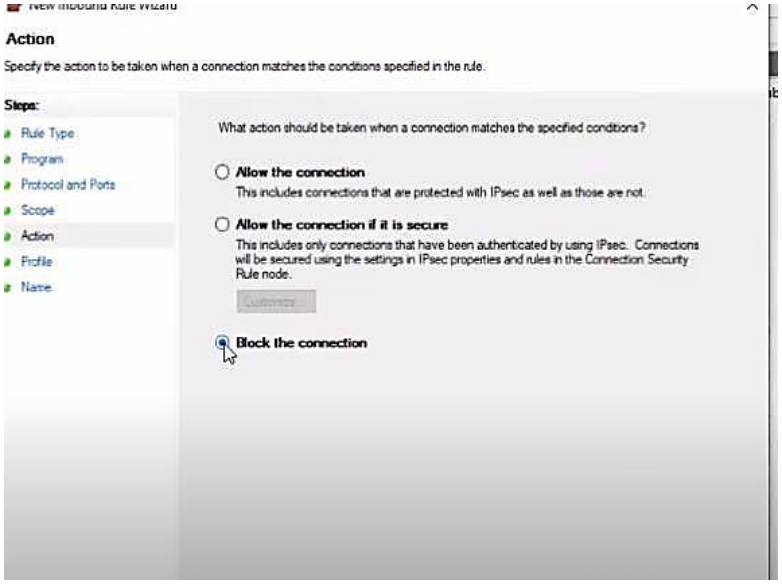
3. We open the windows Firewall settings and apply the Inbound Rule





4. Insert the IP addresses both IPv4 and IPv6. Then select Block connection





5. Provide a suitable name and finish. Repeat the above for Outbound Rules Now if we try to access the website www.android.com , it would be blocked

